

Regression Metrics

Regression metrics measure how close predicted continuous values are to true continuous values. They are used in house-price prediction, demand forecasting, risk scoring, delivery-time estimation, energy prediction, recommendation ranking scores, and many other ML systems where the output is numeric.

This guide covers:

Metric	Full Name	Best For
MAE	Mean Absolute Error	Robust, easy-to-explain error in original units
MSE	Mean Squared Error	Penalizing large errors heavily
RMSE	Root Mean Squared Error	MSE-style penalty in original units
R ² score	Coefficient of Determination	Explaining variance relative to a baseline
Adjusted R ²	Penalized R ²	Comparing models with different feature counts
MAPE	Mean Absolute Percentage Error	Percentage error when target is positive and non-zero
Huber loss	Smooth robust loss	Training robust regression models
Quantile loss	Pinball loss	Predicting quantiles and uncertainty intervals

General notation:

- y_i : true target value for sample i
- $\hat{y}_i$: predicted value for sample i
- n : number of samples
- p : number of predictors/features
- $\text{mean}(y)$: average of true target values
- $e_i = y_i - \hat{y}_i$: residual/error

MAE

1. Overview

MAE, or Mean Absolute Error, measures the average absolute difference between actual and predicted values. It is one of the simplest and most interpretable regression metrics because the final value is in the same unit as the target.

If a house-price model has MAE = 25000, it means the model is wrong by about 25000 currency units on average. If a delivery-time model has MAE = 4 minutes, the average absolute mistake is 4 minutes.

MAE is useful in real-world ML systems when stakeholders care about average error size and want a metric that is not overly dominated by a few extreme mistakes.

2. Intuition

MAE asks: "On average, how far away are my predictions from the truth?"

It ignores whether the model overpredicted or underpredicted. A prediction error of **+10** and **-10** both count as **10**.

Example:

Actual	Predicted	Error	Absolute Error
100	90	10	10
200	230	-30	30
300	310	-10	10

$MAE = (10 + 30 + 10) / 3 = 16.67$.

3. Prerequisites

- Regression basics
- Prediction error and residuals
- Mean/average
- Absolute value
- Basic NumPy or scikit-learn usage

4. Core Concepts

Absolute Error

Absolute error is $|y_i - \hat{y}_i|$.

It matters because positive and negative errors should not cancel each other. In interviews, expect the question: "Why not just average raw errors?" The answer: raw errors can cancel out and falsely suggest a good model.

Same Unit as Target

MAE has the same unit as y .

Example: if y is salary in INR, MAE is also in INR. This makes it business-friendly. Interview angle: MAE is often easier to explain to non-technical stakeholders than MSE.

Robustness Compared to MSE

MAE treats every extra unit of error linearly.

An error of **20** is twice as bad as an error of **10**, not four times as bad. This makes MAE less sensitive to outliers than MSE.

5. Algorithm / Working Process

1. Take actual values y .
2. Take predicted values y_{hat} .
3. Compute residuals: $e_i = y_i - y_{\text{hat}_i}$.
4. Convert residuals to absolute values.
5. Average all absolute errors.
6. Lower MAE means better predictions.

Input: actual and predicted numeric arrays.

Output: one non-negative scalar.

Training usage: MAE can be used as a loss function, but because it is not differentiable at zero, gradient-based optimization may be less smooth than MSE.

Inference usage: after generating predictions, compute MAE on validation/test data.

6. Mathematical Foundation

Formula:

$$\text{MAE} = (1/n) * \sum(|y_i - y_{\text{hat}_i}|)$$

Properties:

- Minimum value is 0.
- Lower is better.
- Unit is the same as the target.
- Optimizing MAE estimates the conditional median, not the conditional mean.

Why median? For a constant prediction, the value that minimizes absolute error is the median of the target distribution.

7. Practical Implementation

```
import numpy as np
from sklearn.metrics import mean_absolute_error

y_true = np.array([100, 200, 300, 400])
y_pred = np.array([110, 180, 330, 390])

mae_manual = np.mean(np.abs(y_true - y_pred))
mae_sklearn = mean_absolute_error(y_true, y_pred)

print("Manual MAE:", mae_manual)
print("sklearn MAE:", mae_sklearn)
```

8. Code Explanation

`y_true` stores the actual target values. `y_pred` stores model predictions.

`y_true - y_pred` computes residuals. `np.abs(...)` removes signs. `np.mean(...)` averages the absolute errors.

`mean_absolute_error` is the scikit-learn implementation and should match the manual result.

9. Training / Evaluation

Use MAE on validation and test sets when the target unit has a natural meaning.

Good for:

- House price error
- Delivery-time prediction
- Demand forecasting
- Revenue prediction

Watch train MAE and validation MAE together. If train MAE is low but validation MAE is high, the model is overfitting.

10. Complexity and Cost

Time complexity: $O(n)$.

Memory usage: $O(1)$ if computed in a streaming way, $O(n)$ if storing all errors.

No GPU is required just to compute MAE.

11. Common Use Cases

- Forecasting sales units
- Predicting prices
- Predicting wait time
- Evaluating regression baselines
- Reporting model error to business teams

12. Common Mistakes

- Comparing MAE across datasets with different target scales.
- Forgetting that MAE hides error direction.
- Using MAE when large errors should be punished heavily.
- Computing MAE on training data only.
- Not inverse-transforming scaled targets before metric calculation.

13. Edge Cases / Limitations

- Does not penalize large errors strongly.
- Not differentiable at zero.
- Can look good even if rare but severe errors are unacceptable.
- Hard to compare across different units or target scales.

14. Variations

Variation	What Changes	When to Use	Placement Importance
Weighted MAE	Errors get sample weights	Important samples cost more	Medium
Median Absolute Error	Uses median instead of mean	Very outlier-heavy data	Medium
Normalized MAE	Divides MAE by range/mean	Comparing across scales	Medium

15. Related Topics

- MSE: more sensitive to large errors.
- RMSE: square-rooted MSE in original units.
- Median regression: connected because MAE optimization targets the median.
- L1 loss: MAE is the average L1 regression loss.

16. Interview Questions

1. What is MAE? Answer: The average absolute difference between true and predicted values.
2. What is the formula for MAE? Answer: $(1/n) * \sum(|y_i - \hat{y}_i|)$.
3. Why use absolute values? Answer: To prevent positive and negative errors from canceling.
4. Is lower MAE better? Answer: Yes. Best possible MAE is zero.
5. Is MAE sensitive to outliers? Answer: Less sensitive than MSE because it grows linearly.
6. What unit does MAE have? Answer: The same unit as the target variable.
7. When would you prefer MAE over RMSE? Answer: When outliers should not dominate the metric.
8. Can MAE be used as a loss function? Answer: Yes, but it is not differentiable at zero.
9. What constant prediction minimizes MAE? Answer: The median of the target values.
10. What is a common MAE mistake? Answer: Reporting MAE on scaled targets instead of original units.

17. Practice Tasks

- Write MAE from scratch using NumPy.
- Compare MAE and RMSE on a dataset with outliers.
- Train Linear Regression on a housing dataset and report MAE.
- Debug a case where MAE is calculated before inverse scaling.
- Add sample weights to MAE manually.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
House Price Error Dashboard	Reports MAE by city and property type	Python, Pandas, sklearn	Kaggle housing data	Shows metric interpretation
Delivery ETA Predictor	Predicts delivery time in minutes	sklearn, FastAPI	Synthetic/order data	Business-friendly regression
Energy Forecasting	Predicts daily electricity demand	Pandas, XGBoost	UCI energy data	Time-series evaluation

19. Quick Revision

- Key idea: average absolute prediction error.
- Main formula: $(1/n) * \sum(|y_i - \hat{y}_i|)$.
- When to use: when interpretability and robustness matter.
- Important metric behavior: linear penalty.
- Common trap: comparing MAE across different target scales.
- Interview one-liner: "MAE tells average error in the original target unit and is less outlier-sensitive than MSE."

20. Final Cheat Sheet

Item	MAE
Definition	Mean absolute prediction error
Input/output	$y_{\text{true}}, y_{\text{pred}}$ -> scalar error
Main steps	residual -> absolute value -> mean
Key hyperparameters	None
Metrics	Lower is better
Pros	Simple, robust, interpretable
Cons	Does not heavily punish large errors
Best use cases	Business-facing regression error

MSE

1. Overview

MSE, or Mean Squared Error, measures the average squared difference between actual and predicted values. It is widely used in regression training because it is smooth, differentiable, and works naturally with gradient-based optimization.

MSE strongly penalizes large errors. This is useful when a few bad predictions are much more harmful than many small mistakes.

2. Intuition

MSE says: "Large mistakes should hurt much more than small mistakes."

If one prediction is wrong by 10, squared error is 100. If another is wrong by 100, squared error is 10000. The larger error becomes much more influential.

3. Prerequisites

- Regression and residuals
- Squaring numbers
- Mean
- Basic calculus for gradients
- Optimization with gradient descent

4. Core Concepts

Squared Error

Squared error is $(y_i - \hat{y}_i)^2$.

It matters because squaring removes signs and amplifies large errors.

Interview angle: explain why MSE is more outlier-sensitive than MAE.

Smooth Loss

MSE is differentiable everywhere.

This matters for training models with gradient descent. The derivative is simple and stable.

Unit Problem

MSE is in squared target units.

If target is dollars, MSE is dollars squared, which is less intuitive. This is why RMSE is often preferred for reporting.

5. Algorithm / Working Process

1. Compute residuals.
2. Square each residual.
3. Average squared residuals.
4. Lower MSE means better performance.

Training: many regression models minimize MSE directly.

Evaluation: MSE is useful when large errors are especially bad.

6. Mathematical Foundation

Formula:

$$\text{MSE} = (1/n) * \sum((y_i - y_{\text{hat}_i})^2)$$

Gradient with respect to prediction:

$$d\text{MSE}/dy_{\text{hat}_i} = (2/n) * (y_{\text{hat}_i} - y_i)$$

For a constant prediction, MSE is minimized by the mean of y .

Under Gaussian noise assumptions, minimizing MSE is equivalent to maximum likelihood estimation.

7. Practical Implementation

```
import numpy as np
from sklearn.metrics import mean_squared_error

y_true = np.array([3.0, 5.0, 7.0, 9.0])
y_pred = np.array([2.5, 5.5, 8.0, 7.0])

mse_manual = np.mean((y_true - y_pred) ** 2)
mse_sklearn = mean_squared_error(y_true, y_pred)

print("Manual MSE:", mse_manual)
print("sklearn MSE:", mse_sklearn)
```

8. Code Explanation

The residuals are computed using $y_{\text{true}} - y_{\text{pred}}$.

$** 2$ squares every residual, so negative and positive errors both become positive.

`np.mean` averages squared errors. `mean_squared_error` gives the standard scikit-learn version.

9. Training / Evaluation

MSE is common as a training objective for:

- Linear Regression
- Neural network regression
- Autoencoders
- Forecasting models

Use validation MSE to tune model complexity, regularization, learning rate, and feature engineering.

If validation MSE is much higher than train MSE, the model may be overfitting.

10. Complexity and Cost

Time complexity: $O(n)$.

Memory: $O(1)$ streaming or $O(n)$ vectorized.

Computing MSE is cheap. Training cost depends on the model, not the metric itself.

11. Common Use Cases

- Training regression neural networks
- Penalizing large prediction errors
- Measuring reconstruction error in autoencoders
- Model selection when rare large errors matter

12. Common Mistakes

- Reporting MSE to business teams without explaining squared units.
- Using MSE on data with extreme outliers without checking robustness.
- Comparing MSE across differently scaled targets.
- Forgetting that MSE can be dominated by a few large errors.
- Using MSE when median behavior is desired.

13. Edge Cases / Limitations

- Very sensitive to outliers.
- Squared units are hard to interpret.
- Large target scales create very large MSE values.
- Can encourage conservative predictions around the mean.

14. Variations

Variation	What Changes	When to Use	Placement Importance
Weighted MSE	Weights individual squared errors	Important samples deserve more penalty	Medium
RMSE	Square root of MSE	Need original units	High
MSLE	Squared log error	Targets with exponential scale	Medium

15. Related Topics

- RMSE: square root of MSE.
- L2 loss: MSE is average L2 regression loss.
- Gaussian likelihood: MSE matches Gaussian error assumption.
- Ridge regression: also uses L2 penalty, but on weights instead of prediction errors.

16. Interview Questions

1. What is MSE? Answer: Average squared prediction error.
2. What is the formula? Answer: $(1/n) * \sum((y_i - \hat{y}_i)^2)$.
3. Why square errors? Answer: To remove signs and penalize large errors more.
4. Is MSE in original units? Answer: No, it is in squared units.
5. Why is MSE popular for training? Answer: It is smooth and differentiable.
6. Is MSE robust to outliers? Answer: No, it is highly sensitive to outliers.
7. What constant prediction minimizes MSE? Answer: The mean of the target.
8. What noise assumption matches MSE? Answer: Gaussian noise.
9. When should MSE be avoided? Answer: When outliers are common and should not dominate.
10. Difference between MSE and MAE? Answer: MSE squares errors and punishes large errors more; MAE uses absolute errors.

17. Practice Tasks

- Implement MSE manually.
- Add one outlier and compare MAE vs MSE.
- Train a regression model using MSE loss.
- Plot squared error as a function of residual.
- Compare MSE before and after target scaling.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Neural Regression Model	Predicts medical cost	PyTorch, Pandas	Medical cost dataset	Shows training loss usage
Autoencoder Error Detector	Detects anomalies via reconstruction MSE	PyTorch	Credit card/anomaly data	Useful DL project
Forecast Model Benchmark	Compares models by MSE	sklearn, XGBoost	Store sales	Evaluation depth

19. Quick Revision

- Key idea: average squared error.
- Formula: $(1/n) * \sum((y_i - \hat{y}_i)^2)$.
- Use when: large errors are costly.
- Trap: not interpretable in original units.
- Interview one-liner: "MSE is smooth and optimization-friendly, but outlier-sensitive."

20. Final Cheat Sheet

Item	MSE
Definition	Mean squared prediction error
Input/output	Actual and predicted values -> scalar
Main steps	residual -> square -> mean
Key hyperparameters	None
Metrics	Lower is better
Pros	Smooth, penalizes large errors
Cons	Outlier-sensitive, squared units
Best use cases	Model training and high-cost error settings

RMSE

1. Overview

RMSE, or Root Mean Squared Error, is the square root of MSE. It keeps MSE's strong penalty for large errors but brings the metric back to the original target unit.

RMSE is very common in ML competitions, forecasting projects, and regression model reports.

2. Intuition

RMSE is like asking: "What is the typical error size if large mistakes matter more?"

Compared with MAE, RMSE becomes larger when the model makes occasional big mistakes.

3. Prerequisites

- MSE
- Square root
- Residuals
- Mean
- Outlier sensitivity

4. Core Concepts

Root of MSE

RMSE is $\sqrt{\text{MSE}}$.

It matters because the result returns to the target unit.

Large Error Penalty

Since RMSE comes from squared errors, large residuals still dominate.

Interview angle: if RMSE is much larger than MAE, the model likely has large outlier errors.

Interpretability

RMSE is easier to explain than MSE because it has the same unit as the target.

5. Algorithm / Working Process

1. Compute residuals.
2. Square residuals.
3. Average squared residuals.
4. Take the square root.
5. Lower RMSE means better performance.

6. Mathematical Foundation

Formula:

$$\text{RMSE} = \sqrt{\left(\frac{1}{n}\right) * \sum((y_i - \hat{y}_i)^2)}$$

RMSE is monotonic with MSE. If one model has lower MSE, it also has lower RMSE on the same data.

7. Practical Implementation

```
import numpy as np
from sklearn.metrics import mean_squared_error

y_true = np.array([10, 20, 30, 40])
y_pred = np.array([12, 18, 33, 39])

rmse_manual = np.sqrt(np.mean((y_true - y_pred) ** 2))
rmse_sklearn = mean_squared_error(y_true, y_pred, squared=False)

print("Manual RMSE:", rmse_manual)
print("sklearn RMSE:", rmse_sklearn)
```

8. Code Explanation

The manual code first computes MSE using NumPy and then applies `np.sqrt`.

In scikit-learn, `mean_squared_error(..., squared=False)` returns RMSE.

9. Training / Evaluation

RMSE is usually an evaluation metric, while MSE is often used as the training loss.

Use RMSE when:

- Large errors matter.
- You still want original target units.
- You want to compare with leaderboard metrics.

10. Complexity and Cost

Time complexity: $O(n)$.

Memory: $O(1)$ streaming or $O(n)$ vectorized.

Extra square root cost is negligible.

11. Common Use Cases

- Price prediction
- Demand forecasting
- Sensor prediction
- Kaggle regression competitions
- Scientific measurement error

12. Common Mistakes

- Thinking RMSE is robust to outliers.
- Comparing RMSE across different target scales.
- Forgetting that $RMSE \geq MAE$ for the same predictions.
- Reporting RMSE without checking residual distribution.

13. Edge Cases / Limitations

- Sensitive to outliers.
- Can overemphasize rare large errors.
- Not ideal when business cost grows linearly.
- Can hide systematic bias because errors are squared.

14. Variations

Variation	What Changes	When to Use	Placement Importance
Normalized RMSE	Divides RMSE by range/mean/std	Compare across datasets	Medium
RMSLE	Uses log targets	Relative errors matter more	Medium
Weighted RMSE	Weights samples	Some samples are more costly	Medium

15. Related Topics

- MSE: RMSE is square root of MSE.
- MAE: compare with RMSE to detect large errors.
- Standard deviation: RMSE resembles residual standard deviation.
- Gaussian regression: RMSE is natural under squared-error assumptions.

16. Interview Questions

1. What is RMSE? Answer: Square root of mean squared error.
2. Formula? Answer: $\sqrt{(1/n) * \sum((y_i - \hat{y}_i)^2)}$.
3. Why use RMSE instead of MSE? Answer: RMSE is in original target units.
4. Is RMSE sensitive to outliers? Answer: Yes.
5. Can RMSE be lower than MAE? Answer: No, not for the same error set.
6. What does high RMSE but low MAE suggest? Answer: A few large errors.
7. Is RMSE differentiable? Answer: Mostly yes, but MSE is simpler for optimization.
8. When is RMSE preferred? Answer: When large errors are costly and interpretability matters.
9. Does lower RMSE always mean better business value? Answer: Not always; it depends on the cost function.
10. How do you reduce RMSE? Answer: Improve features, handle outliers, tune model, reduce large residuals.

17. Practice Tasks

- Compute RMSE manually.
- Compare RMSE and MAE after adding outliers.
- Plot residual histogram and explain RMSE behavior.
- Use RMSE in cross-validation.
- Build a model leaderboard sorted by RMSE.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Sales Forecast Leaderboard	Compares models by RMSE	sklearn, XGBoost	Store sales	Practical evaluation
Sensor Calibration	Predicts corrected sensor values	NumPy, sklearn	Sensor data	Engineering relevance
Housing RMSE Benchmark	Builds regression pipeline	Pandas, sklearn	Ames housing	Interview-ready project

19. Quick Revision

- Key idea: square-rooted MSE.
- Formula: $\sqrt{\text{mean squared error}}$.
- Use when: large errors matter and original units help.
- Trap: outlier sensitivity.
- Interview one-liner: "RMSE is MSE converted back to the target unit."

20. Final Cheat Sheet

Item	RMSE
Definition	Root mean squared prediction error
Input/output	$y_{\text{true}}, y_{\text{pred}}$ -> scalar
Main steps	residual -> square -> mean -> root
Key hyperparameters	None
Metrics	Lower is better
Pros	Original units, punishes large errors
Cons	Outlier-sensitive
Best use cases	Forecasting and regression reports

R² Score

1. Overview

R² score, or coefficient of determination, measures how much variance in the target is explained by the model compared with a simple baseline that always predicts the mean target value.

Unlike MAE, MSE, and RMSE, R² is a relative goodness-of-fit metric, not an error in original units.

2. Intuition

R² asks: "How much better is my model than predicting the average every time?"

If R² = 0.80, the model explains about 80% of the target variance compared with the mean baseline.

3. Prerequisites

- Mean
- Variance
- Residual sum of squares
- Baseline model
- Train/test evaluation

4. Core Concepts

Baseline Mean Model

The baseline predicts $\text{mean}(y)$ for every sample.

This matters because R^2 compares your model against a naive but meaningful benchmark.

Residual Sum of Squares

RSS measures unexplained error:

$$\text{RSS} = \sum (y_i - \hat{y}_i)^2$$

Total Sum of Squares

TSS measures total target variation:

$$\text{TSS} = \sum (y_i - \text{mean}(y))^2$$

Score Interpretation

- $R^2 = 1$: perfect prediction.
- $R^2 = 0$: same as mean baseline.
- $R^2 < 0$: worse than mean baseline.

5. Algorithm / Working Process

1. Compute mean of actual target values.
2. Compute RSS from model predictions.
3. Compute TSS from target values and their mean.
4. Calculate $1 - \text{RSS}/\text{TSS}$.
5. Higher R^2 is better.

6. Mathematical Foundation

Formula:

$$R^2 = 1 - \text{RSS}/\text{TSS}$$

$$\text{RSS} = \sum (y_i - \hat{y}_i)^2$$

$$\text{TSS} = \sum (y_i - \text{mean}(y))^2$$

R^2 measures relative reduction in squared error compared with the mean baseline.

7. Practical Implementation


```
import numpy as np
from sklearn.metrics import r2_score

y_true = np.array([10, 20, 30, 40, 50])
y_pred = np.array([12, 19, 29, 43, 48])

rss = np.sum((y_true - y_pred) ** 2)
tss = np.sum((y_true - np.mean(y_true)) ** 2)
r2_manual = 1 - (rss / tss)
r2_sklearn = r2_score(y_true, y_pred)

print("Manual R2:", r2_manual)
print("sklearn R2:", r2_sklearn)
```

8. Code Explanation

`rss` calculates model error. `tss` calculates how much the target varies around its mean.

`1 - rss/tss` gives improvement over the mean baseline.

`r2_score` is the standard scikit-learn implementation.

9. Training / Evaluation

R^2 is usually used for evaluation, not as a direct training loss.

Use it with MAE/RMSE because R^2 alone does not tell error size in target units.

High train R^2 and low test R^2 indicates overfitting.

10. Complexity and Cost

Time complexity: $O(n)$.

Memory: $O(1)$ if streamed.

No GPU required.

11. Common Use Cases

- Linear regression model reporting
- Comparing regression models on the same dataset
- Explaining model fit in statistics
- Academic and placement interviews

12. Common Mistakes

- Thinking R^2 means accuracy percentage.
- Comparing R^2 across different datasets without context.
- Assuming high R^2 means causation.

- Ignoring negative R^2 .
- Using train R^2 only.

13. Edge Cases / Limitations

- Undefined or unstable when target variance is zero.
- Can be negative.
- Does not show absolute error size.
- Can increase when adding useless features in ordinary least squares.
- Not always meaningful for non-linear or non-standard regression settings.

14. Variations

Variation	What Changes	When to Use	Placement Importance
Adjusted R^2	Penalizes number of predictors	Feature-count comparison	High
Cross-validated R^2	Average across folds	More reliable evaluation	High
Pseudo R^2	For generalized models	Logistic/GLM settings	Medium

15. Related Topics

- Adjusted R^2 : fixes feature-count inflation.
- MSE: R^2 is based on squared error.
- ANOVA: decomposition of variance.
- Linear regression: R^2 is a standard fit statistic.

16. Interview Questions

1. What does R^2 measure? Answer: Variance explained by the model relative to the mean baseline.
2. Formula? Answer: $1 - \text{RSS}/\text{TSS}$.
3. What is RSS? Answer: Sum of squared residuals.
4. What is TSS? Answer: Total variation of target around its mean.
5. Can R^2 be negative? Answer: Yes, if the model is worse than predicting the mean.
6. Is R^2 accuracy? Answer: No.
7. What does $R^2 = 0$ mean? Answer: Same performance as mean baseline.
8. What does $R^2 = 1$ mean? Answer: Perfect predictions.
9. Why use adjusted R^2 ? Answer: To penalize unnecessary predictors.
10. Should R^2 be used alone? Answer: No, pair it with error metrics like MAE or RMSE.

17. Practice Tasks

- Implement R^2 manually.
- Create a bad model with negative R^2 .
- Compare train/test R^2 for overfitting.
- Add random features and observe R^2 .
- Report R^2 with MAE and RMSE.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Regression Fit Analyzer	Reports R^2 , residuals, plots	sklearn, Matplotlib	Any tabular dataset	Strong interview utility
Feature Impact Study	Tracks R^2 after adding features	Pandas, sklearn	Housing data	Shows feature reasoning
Model Comparison App	Compares Linear, RF, XGBoost	Streamlit, sklearn	UCI datasets	Portfolio-ready

19. Quick Revision

- Key idea: variance explained compared with mean baseline.
- Formula: $R^2 = 1 - \text{RSS}/\text{TSS}$.
- Use when: comparing model fit on same target/data.
- Trap: treating it as accuracy.
- Interview one-liner: " R^2 measures improvement over predicting the target mean."

20. Final Cheat Sheet

Item	R^2
Definition	Coefficient of determination
Input/output	Actual and predicted values -> score
Main steps	compute RSS and TSS
Key hyperparameters	None
Metrics	Higher is better
Pros	Baseline-relative, common
Cons	Can mislead, can be negative
Best use cases	Regression model fit reporting

Adjusted R^2

1. Overview

Adjusted R^2 modifies R^2 by penalizing the number of predictors in the model. It is mainly used in regression analysis when comparing models with different numbers of features.

Ordinary R^2 often increases when more features are added, even if those features are useless. Adjusted R^2 reduces this problem.

2. Intuition

Adjusted R^2 asks: "Did this new feature improve the model enough to justify its existence?"

If a feature adds real predictive power, adjusted R^2 can increase. If it only adds noise, adjusted R^2 can decrease.

3. Prerequisites

- R^2
- Number of samples
- Number of predictors
- Overfitting
- Degrees of freedom

4. Core Concepts

Feature Penalty

Adjusted R^2 penalizes model complexity through p , the number of predictors.

This matters because adding features can make a model look better on training data without improving generalization.

Degrees of Freedom

The formula accounts for $n - p - 1$.

This matters because models with many predictors have less freedom left to estimate error reliably.

Model Comparison

Adjusted R^2 is useful for comparing linear regression models with different feature counts.

Interview angle: it is not a replacement for validation/test evaluation.

5. Algorithm / Working Process

1. Compute ordinary R^2 .
2. Count number of samples n .
3. Count number of predictors p .
4. Apply adjusted R^2 formula.
5. Higher adjusted R^2 is better.

6. Mathematical Foundation

Formula:

$$\text{Adjusted } R^2 = 1 - ((1 - R^2) * (n - 1) / (n - p - 1))$$

Where:

- n : number of observations
- p : number of predictors

Requirement:

$$n > p + 1$$

If p is too large relative to n , adjusted R^2 becomes unstable or undefined.

7. Practical Implementation

```
import numpy as np
from sklearn.metrics import r2_score

def adjusted_r2_score(y_true, y_pred, num_features):
    n = len(y_true)
    if n <= num_features + 1:
        raise ValueError("Adjusted R2 requires n > p + 1")

    r2 = r2_score(y_true, y_pred)
    return 1 - ((1 - r2) * (n - 1) / (n - num_features - 1))

y_true = np.array([100, 150, 200, 250, 300, 350])
y_pred = np.array([110, 145, 195, 260, 290, 360])

print("Adjusted R2:", adjusted_r2_score(y_true, y_pred, num_features=2))
```

8. Code Explanation

The function first checks that sample size is large enough for the number of predictors.

It computes ordinary R^2 using scikit-learn, then applies the adjusted R^2 formula.

`num_features` should be the number of input predictors used by the model.

9. Training / Evaluation

Adjusted R^2 is mainly a model-selection statistic.

Use it when:

- Comparing linear models with different feature sets.
- Doing feature selection.
- Explaining whether extra variables are useful.

Still evaluate final performance on validation/test data using MAE/RMSE.

10. Complexity and Cost

Time complexity: $O(n)$ because it depends on R^2 .

Memory: $O(1)$ or $O(n)$ depending on implementation.

No extra training cost.

11. Common Use Cases

- Feature selection in regression
- Comparing statistical linear models
- Placement interview questions on R^2 limitations
- Regression reports in analytics

12. Common Mistakes

- Using adjusted R^2 with $p \geq n - 1$.
- Thinking adjusted R^2 proves causal importance.
- Using it instead of test-set evaluation.
- Counting one-hot encoded features incorrectly.
- Applying it blindly to complex ML models.

13. Edge Cases / Limitations

- Not very useful for black-box non-linear models.
- Can be unstable with small datasets.
- Depends on correct feature count.
- Does not replace cross-validation.

14. Variations

Variation	What Changes	When to Use	Placement Importance
AIC	Penalizes likelihood by parameters	Statistical model comparison	Medium
BIC	Stronger complexity penalty	Simpler model preference	Medium
Cross-validated R^2	Uses validation folds	ML model selection	High

15. Related Topics

- R^2 : adjusted R^2 starts from R^2 .
- Feature selection: adjusted R^2 helps judge feature usefulness.
- Overfitting: penalty discourages useless predictors.
- AIC/BIC: alternative complexity-aware criteria.

16. Interview Questions

1. What is adjusted R^2 ? Answer: R^2 modified to penalize the number of predictors.
2. Formula? Answer: $1 - ((1 - R^2)(n - 1)/(n - p - 1))$.
3. Why is it needed? Answer: Ordinary R^2 can increase after adding useless features.
4. Can adjusted R^2 decrease? Answer: Yes, when new features do not help enough.
5. What are n and p ? Answer: n is sample count; p is predictor count.
6. When is adjusted R^2 invalid? Answer: When $n \leq p + 1$.
7. Is adjusted R^2 better than cross-validation? Answer: No, cross-validation is usually better for ML generalization.
8. Does adjusted R^2 handle overfitting? Answer: It helps penalize complexity but does not fully prevent overfitting.
9. Should one-hot columns count in p ? Answer: Yes, count actual model predictors.
10. Where is adjusted R^2 most common? Answer: Statistical linear regression.

17. Practice Tasks

- Compute adjusted R^2 from R^2 manually.
- Add random noise features and observe adjusted R^2 .
- Compare adjusted R^2 with test RMSE.
- Build forward feature selection using adjusted R^2 .
- Test what happens when p approaches n .

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Feature Selection Report	Ranks features by adjusted R^2 improvement	sklearn, Pandas	Housing	Strong analytics story
Linear Model Comparison	Compares multiple formulas	statsmodels	Advertising/sales	Statistics depth
Overfitting Demo	Shows R^2 vs adjusted R^2 with noise	NumPy, Matplotlib	Synthetic	Interview teaching project

19. Quick Revision

- Key idea: R^2 with feature-count penalty.
- Formula: $1 - ((1 - R^2)(n - 1)/(n - p - 1))$.
- Use when: comparing feature sets.
- Trap: using it as a test-set substitute.
- Interview one-liner: "Adjusted R^2 rewards useful features and penalizes unnecessary ones."

20. Final Cheat Sheet

Item	Adjusted R^2
Definition	Complexity-penalized R^2
Input/output	R^2 , n , $p \rightarrow$ score
Main steps	compute R^2 , apply penalty
Key hyperparameters	Number of predictors
Metrics	Higher is better
Pros	Penalizes useless features
Cons	Less useful for complex ML models
Best use cases	Linear model comparison

MAPE

1. Overview

MAPE, or Mean Absolute Percentage Error, measures average absolute error as a percentage of the true value. It is popular in business forecasting because percentages are easy to understand.

Example: MAPE = 8% means predictions are off by about 8% on average.

2. Intuition

MAPE asks: "How large is the error relative to the actual value?"

An error of 10 is huge if actual value is 20, but small if actual value is 1000. MAPE captures this relative scale.

3. Prerequisites

- MAE
- Percentages
- Division and ratios
- Handling zero values
- Forecast evaluation

4. Core Concepts

Percentage Error

Percentage error is:

$$|y_i - \hat{y}_i| / |y_i|$$

It matters because it normalizes error by target size.

Business Interpretability

MAPE is easy to communicate: "average percentage error."

Interview angle: it is intuitive but dangerous when actual values are zero or near zero.

Scale Independence

MAPE is unitless, so it can compare errors across similar business segments.

But it is not safe when targets can be zero, negative, or very small.

5. Algorithm / Working Process

1. Compute absolute error.
2. Divide by absolute actual value.
3. Average all percentage errors.
4. Multiply by 100 if reporting as percent.

6. Mathematical Foundation

Formula:

$$\text{MAPE} = (100/n) * \sum(|y_i - \hat{y}_i| / |y_i|)$$

Usually requires:

$$y_i \neq 0$$

MAPE heavily penalizes errors where actual values are small.

7. Practical Implementation

```
import numpy as np
```

```
def mean_absolute_percentage_error(y_true, y_pred, epsilon=1e-8):
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)

    denominator = np.maximum(np.abs(y_true), epsilon)
    return np.mean(np.abs((y_true - y_pred) / denominator)) * 100

y_true = np.array([100, 200, 400, 800])
y_pred = np.array([110, 180, 420, 760])

print("MAPE (%):", mean_absolute_percentage_error(y_true, y_pred))
```

8. Code Explanation

The function converts inputs to float arrays.

`denominator` avoids division by exact zero using `epsilon`.

The absolute percentage errors are averaged and multiplied by 100.

In production, do not silently hide zero-target issues. Decide whether zero values should be excluded, clipped, or evaluated using another metric.

9. Training / Evaluation

MAPE is usually an evaluation metric for forecasting, not the primary training loss.

Use it when:

- Targets are positive.
- Targets are not close to zero.
- Relative error matters more than absolute error.

Always inspect zero and near-zero target values before reporting MAPE.

10. Complexity and Cost

Time complexity: $O(n)$.

Memory: $O(n)$ vectorized or $O(1)$ streaming.

No special hardware required.

11. Common Use Cases

- Sales forecasting
- Demand planning
- Inventory forecasting
- Revenue prediction
- Business KPI forecasting

12. Common Mistakes

- Using MAPE when y_{true} contains zeros.
- Ignoring huge MAPE from small actual values.
- Using MAPE with negative targets without careful definition.
- Thinking MAPE is symmetric. Overprediction and underprediction can behave differently.
- Reporting MAPE without sample count or target distribution.

13. Edge Cases / Limitations

- Undefined at $y_i = 0$.
- Explodes near zero.
- Biased toward under-forecasting in some settings.
- Not suitable for data with negative true values.
- Can be misleading for intermittent demand.

14. Variations

Variation	What Changes	When to Use	Placement Importance
SMAPE	Divides by average of actual and predicted magnitudes	Reduces zero issue somewhat	Medium
WMAPE	Ratio of total absolute error to total actual	Business forecasting	High
RMSPE	Squares percentage error	Large relative errors matter	Medium

15. Related Topics

- MAE: MAPE is like relative MAE.
- WMAPE: more stable for aggregated business forecasting.
- SMAPE: attempts symmetry.
- Time-series forecasting: MAPE is commonly used there.

16. Interview Questions

1. What is MAPE? Answer: Mean absolute percentage error.
2. Formula? Answer: $(100/n) * \sum(|(y_i - \hat{y}_i)/y_i|)$.
3. Why is MAPE popular? Answer: Percentages are easy for business users.
4. Biggest limitation? Answer: It fails or explodes when actual values are zero or near zero.
5. Is MAPE unitless? Answer: Yes.
6. Can MAPE be used for negative targets? Answer: Usually not without careful handling.
7. When is WMAPE better? Answer: When evaluating aggregate forecast error.
8. What does MAPE = 10 mean? Answer: Average absolute percentage error is about 10%.

9. Does MAPE penalize all absolute errors equally? Answer: No, it scales them by actual value.

10. Should MAPE be used alone? Answer: No, pair it with MAE/RMSE and check zero targets.

17. Practice Tasks

- Implement MAPE with zero handling.
- Compare MAPE and MAE on high-value and low-value targets.
- Find rows causing extreme MAPE.
- Calculate WMAPE for a sales dataset.
- Build a forecast report with MAPE by product category.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Sales Forecast Evaluator	Reports MAPE/WMAPE by SKU	Pandas, sklearn	M5 forecasting	Business metric skill
Demand Alert System	Flags high percentage forecast errors	Python, Streamlit	Retail data	Practical dashboard
Forecast Metric Lab	Compares MAPE, SMAPE, WMAPE	NumPy, Matplotlib	Synthetic + real	Interview depth

19. Quick Revision

- Key idea: average absolute percentage error.
- Formula: $(100/n) * \sum(|\text{error}| / \text{actual})$.
- Use when: positive non-zero targets and relative error matters.
- Trap: zero or near-zero actual values.
- Interview one-liner: "MAPE is intuitive but fragile around zero."

20. Final Cheat Sheet

Item	MAPE
Definition	Mean absolute percentage error
Input/output	Actual and predicted values -> percentage
Main steps	absolute error -> divide by actual -> mean
Key hyperparameters	Zero-handling epsilon/policy
Metrics	Lower is better
Pros	Easy business interpretation
Cons	Breaks near zero
Best use cases	Positive-valued forecasting

Huber Loss

1. Overview

Huber loss is a regression loss function that behaves like MSE for small errors and like MAE for large errors. It gives a balance between smooth optimization and robustness to outliers.

It is used in robust regression, deep learning regression, reinforcement learning value estimation, and settings where noisy labels or outliers exist.

2. Intuition

Huber loss says: "For small mistakes, use squared error to learn smoothly. For big mistakes, do not let them dominate training."

It acts like MSE near zero and MAE far away from zero.

3. Prerequisites

- MAE
- MSE
- Piecewise functions
- Gradient descent
- Outliers
- Loss functions

4. Core Concepts

Delta Threshold

Huber loss uses a threshold δ .

Small errors with $|e| \leq \delta$ use squared loss. Large errors with $|e| > \delta$ use linear loss.

Interview angle: δ controls the switch between MSE-like and MAE-like behavior.

Robustness

Large outliers receive linear penalty instead of quadratic penalty.

This prevents a few extreme points from dominating the model.

Smoothness

Unlike MAE, Huber loss is differentiable at zero and smoother for optimization.

5. Algorithm / Working Process

1. Compute residual $e = y - \hat{y}$.
2. Compute absolute residual $|e|$.

3. If $|e| \leq \delta$, use $0.5 * e^2$.
4. If $|e| > \delta$, use $\delta * (|e| - 0.5 * \delta)$.
5. Average across samples.

Training: Huber loss can be optimized with gradient descent.

Evaluation: it is less commonly reported to stakeholders than MAE/RMSE.

6. Mathematical Foundation

Formula:

```
L_delta(e) =
    0.5 * e^2           if |e| <= delta
    delta * (|e| - 0.5 * delta) otherwise
```

Where:

```
e = y_i - y_hat_i
```

Gradient with respect to prediction:

```
dL/dy_hat =
    -e           if |e| <= delta
    -delta * sign(e) otherwise
```

7. Practical Implementation

```
import numpy as np
from sklearn.linear_model import HuberRegressor
from sklearn.metrics import mean_absolute_error

def huber_loss(y_true, y_pred, delta=1.0):
    error = y_true - y_pred
    abs_error = np.abs(error)
    quadratic = 0.5 * error ** 2
    linear = delta * (abs_error - 0.5 * delta)
    return np.mean(np.where(abs_error <= delta, quadratic, linear))

X = np.array([[1], [2], [3], [4], [5], [20]], dtype=float)
y = np.array([2, 4, 6, 8, 10, 100], dtype=float)

model = HuberRegressor(epsilon=1.35)
model.fit(X, y)
pred = model.predict(X)
```

```
print("Huber loss:", huber_loss(y, pred, delta=1.0))
print("MAE:", mean_absolute_error(y, pred))
```

8. Code Explanation

`huber_loss` implements the piecewise formula using `np.where`.

For small errors, it uses the quadratic expression. For large errors, it uses the linear expression.

`HuberRegressor` is a robust scikit-learn model that reduces the effect of outliers compared with ordinary least squares.

9. Training / Evaluation

Use Huber loss during training when:

- Labels contain outliers.
- MSE is too sensitive.
- MAE optimization is less smooth than desired.

Tune `delta` or equivalent parameters using validation data.

Evaluate final model with MAE, RMSE, and residual analysis.

10. Complexity and Cost

Metric computation: $O(n)$.

Training cost depends on the model. Huber loss adds minimal overhead compared with MSE.

Memory usage is similar to MAE/MSE.

11. Common Use Cases

- Robust house price prediction
- Sensor readings with occasional spikes
- Noisy medical measurements
- Reinforcement learning value loss
- Deep regression with label noise

12. Common Mistakes

- Not tuning `delta`.
- Using Huber loss but reporting only training loss.
- Assuming it removes outliers automatically.
- Using too small `delta`, making training MAE-like too early.
- Using too large `delta`, making it almost MSE.

13. Edge Cases / Limitations

- Requires choosing **delta**.
- If outliers are meaningful rare events, reducing their influence may be harmful.
- Less interpretable to non-technical stakeholders.
- Not a full substitute for data cleaning.

14. Variations

Variation	What Changes	When to Use	Placement Importance
Smooth L1 Loss	Common DL version of Huber-like loss	PyTorch models	High
Pseudo-Huber Loss	Smooth approximation everywhere	Optimization-sensitive cases	Medium
Adaptive Huber	Learns/adjusts threshold	Research/robust learning	Low-Medium

15. Related Topics

- MAE: Huber behaves like MAE for large errors.
- MSE: Huber behaves like MSE for small errors.
- Robust regression: Huber is a core robust loss.
- Outlier detection: related but not the same as downweighting outliers.

16. Interview Questions

1. What is Huber loss? Answer: A piecewise loss that is quadratic for small errors and linear for large errors.
2. Why use it? Answer: It combines smooth optimization with outlier robustness.
3. What does **delta** do? Answer: It controls the error threshold where loss changes from quadratic to linear.
4. How is it different from MSE? Answer: Large errors are penalized linearly instead of quadratically.
5. How is it different from MAE? Answer: It is smoother near zero.
6. Is Huber loss differentiable? Answer: Yes, it is differentiable at the transition point.
7. When is it useful? Answer: Noisy regression with outliers.
8. What happens if delta is very large? Answer: Huber behaves like MSE for most errors.
9. What happens if delta is very small? Answer: Huber behaves more like MAE.
10. Is Huber a metric or loss? Answer: Mostly used as a training loss, though it can be computed as an evaluation value.

17. Practice Tasks

- Plot MAE, MSE, and Huber loss against residual size.
- Train Linear Regression and HuberRegressor on outlier data.
- Tune Huber threshold using validation error.
- Implement Huber loss in PyTorch.
- Analyze residuals before and after robust training.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
Robust Price Predictor	Handles outlier property prices	sklearn	Housing data	Shows robust modeling
Sensor Spike Regression	Predicts signal despite spikes	NumPy, sklearn	Sensor dataset	Real-world noise handling
PyTorch Huber Trainer	Trains neural regression with SmoothL1Loss	PyTorch	Tabular/synthetic	DL loss understanding

19. Quick Revision

- Key idea: MSE for small errors, MAE for large errors.
- Formula: piecewise quadratic/linear.
- Use when: outliers exist but smooth training is needed.
- Trap: wrong **delta**.
- Interview one-liner: "Huber loss is a robust compromise between MAE and MSE."

20. Final Cheat Sheet

Item	Huber Loss
Definition	Piecewise robust regression loss
Input/output	Error values -> scalar loss
Main steps	check error against delta
Key hyperparameters	delta / epsilon
Metrics	Lower is better
Pros	Smooth and robust
Cons	Needs threshold tuning
Best use cases	Noisy regression training

Quantile Loss

1. Overview

Quantile loss, also called pinball loss, trains models to predict a chosen quantile of the target distribution instead of only the mean. It is useful when we need uncertainty-aware predictions such as pessimistic, median, or optimistic forecasts.

For example, a delivery system may predict:

- 10th percentile ETA: optimistic arrival time
- 50th percentile ETA: median arrival time
- 90th percentile ETA: conservative arrival time

2. Intuition

Quantile loss says: "Underprediction and overprediction should have different costs."

For the 90th percentile, underpredicting is punished more because the model should produce a value that actual outcomes fall below about 90% of the time.

3. Prerequisites

- Percentiles and quantiles
- Regression
- Residuals
- Asymmetric loss
- Prediction intervals
- Basic probability

4. Core Concepts

Quantile

A quantile q is a value below which approximately q fraction of observations fall.

Example: the 0.9 quantile is the 90th percentile.

Asymmetric Penalty

Quantile loss penalizes overprediction and underprediction differently.

This matters because some business problems have asymmetric risk.

Prediction Intervals

Training models for lower and upper quantiles can create prediction intervals.

Example: train one model for $q=0.1$ and another for $q=0.9$; the interval estimates an 80% range.

5. Algorithm / Working Process

1. Choose quantile q , where $0 < q < 1$.
2. Compute residual $e = y - \hat{y}$.
3. If prediction is too low ($e \geq 0$), loss is $q * e$.

4. If prediction is too high ($e < 0$), loss is $(q - 1) * e$.
5. Average loss over samples.

Training: optimize quantile loss for selected quantile.

Inference: output the predicted quantile value.

6. Mathematical Foundation

Formula:

$$L_q(y, y_{\hat{}}) = \max(q * (y - y_{\hat{}}), (q - 1) * (y - y_{\hat{}}))$$

Equivalent:

$$\begin{aligned}
 e &= y - y_{\hat{}} \\
 L_q(e) &= \begin{cases} q * e & \text{if } e \geq 0 \\ (q - 1) * e & \text{if } e < 0 \end{cases}
 \end{aligned}$$

For $q = 0.5$, quantile loss becomes proportional to MAE and estimates the median.

7. Practical Implementation

```
import numpy as np
from sklearn.ensemble import GradientBoostingRegressor

def quantile_loss(y_true, y_pred, q):
    error = y_true - y_pred
    return np.mean(np.maximum(q * error, (q - 1) * error))

X = np.arange(1, 21).reshape(-1, 1)
y = np.array([
    3, 5, 7, 8, 11, 13, 14, 16, 19, 20,
    23, 25, 26, 28, 30, 35, 37, 38, 40, 45
])

lower_model = GradientBoostingRegressor(loss="quantile", alpha=0.1,
random_state=42)
median_model = GradientBoostingRegressor(loss="quantile", alpha=0.5,
random_state=42)
upper_model = GradientBoostingRegressor(loss="quantile", alpha=0.9,
random_state=42)

lower_model.fit(X, y)
median_model.fit(X, y)
upper_model.fit(X, y)
```

```
x_new = np.array([[21]])  
print("P10:", lower_model.predict(x_new)[0])  
print("P50:", median_model.predict(x_new)[0])  
print("P90:", upper_model.predict(x_new)[0])
```

8. Code Explanation

`quantile_loss` implements pinball loss using `np.maximum`.

`GradientBoostingRegressor(loss="quantile", alpha=q)` trains a model for the selected quantile.

Three models are trained for lower, median, and upper estimates. Together, they give an uncertainty band.

9. Training / Evaluation

Use quantile loss when point predictions are not enough.

Evaluate:

- Pinball loss for each quantile.
- Coverage: how often actual values fall inside predicted intervals.
- Interval width: narrower intervals are better if coverage is maintained.

Avoid using only RMSE if the goal is uncertainty estimation.

10. Complexity and Cost

Metric computation: $O(n)$.

Training cost depends on model choice.

If predicting multiple quantiles with separate models, cost grows roughly linearly with the number of quantiles.

11. Common Use Cases

- Demand forecasting intervals
- ETA uncertainty
- Energy load prediction
- Risk estimation
- Financial forecasting
- Inventory safety stock planning

12. Common Mistakes

- Treating quantile predictions like mean predictions.
- Forgetting that $q=0.5$ predicts median, not mean.
- Training lower and upper quantiles that cross each other.
- Reporting intervals without coverage.
- Using quantile loss when the business only needs average error.

13. Edge Cases / Limitations

- Quantile crossing can happen when separate models are trained.
- More quantiles require more training.
- Does not give a full probability distribution by itself.
- Poor calibration can make intervals unreliable.

14. Variations

Variation	What Changes	When to Use	Placement Importance
Median regression	Uses $q=0.5$	Robust central prediction	High
Multi-quantile regression	Predicts many quantiles	Uncertainty bands	Medium
Conformal prediction	Calibrates intervals	Reliable coverage	Medium-High
Distributional regression	Predicts full distribution	Research/advanced systems	Medium

15. Related Topics

- MAE: quantile loss at $q=0.5$ is related to MAE.
- Prediction intervals: quantiles can form lower/upper bounds.
- Calibration: checks whether predicted intervals match actual frequency.
- Risk-sensitive ML: asymmetric penalties model different costs.

16. Interview Questions

1. What is quantile loss? Answer: An asymmetric loss used to predict a selected quantile.
2. Why is it called pinball loss? Answer: Its graph has asymmetric linear slopes like a pinball loss shape.
3. Formula? Answer: $\max(q(y - \hat{y}), (q - 1)(y - \hat{y}))$.
4. What does $q=0.5$ predict? Answer: The median.
5. Why use $q=0.9$? Answer: To predict an upper/conservative estimate.
6. What is asymmetric about it? Answer: Underprediction and overprediction have different penalties.
7. How create prediction intervals? Answer: Train lower and upper quantile models, such as 0.1 and 0.9.
8. What is quantile crossing? Answer: Lower predicted quantile becomes greater than upper predicted quantile.
9. How evaluate quantile models? Answer: Pinball loss, interval coverage, and interval width.
10. When is quantile loss better than MSE? Answer: When uncertainty or asymmetric risk matters.

17. Practice Tasks

- Implement quantile loss from scratch.
- Plot quantile loss for $q=0.1$, 0.5 , and 0.9 .
- Train three quantile regression models.
- Check interval coverage on test data.
- Fix quantile crossing by sorting predicted quantiles.

18. Project Ideas

Project	What It Does	Tech Stack	Dataset	Resume Value
ETA Prediction Intervals	Predicts lower/median/upper delivery time	sklearn, FastAPI	Delivery/synthetic data	Uncertainty-aware ML
Demand Safety Stock	Uses upper quantile forecasts for inventory	Pandas, sklearn	Retail data	Business impact
Energy Risk Forecast	Predicts 10th/50th/90th demand quantiles	XGBoost/sklearn	Energy data	Advanced forecasting

19. Quick Revision

- Key idea: predict quantiles, not just average values.
- Formula: $\max(qe, (q - 1)e)$.
- Use when: asymmetric risk or uncertainty intervals matter.
- Trap: ignoring quantile crossing and coverage.
- Interview one-liner: "Quantile loss trains models to predict percentiles using asymmetric penalties."

20. Final Cheat Sheet

Item	Quantile Loss
Definition	Asymmetric loss for quantile prediction
Input/output	Actual, predicted, quantile -> scalar loss
Main steps	compute residual, apply asymmetric penalty
Key hyperparameters	Quantile q
Metrics	Pinball loss, coverage, interval width
Pros	Supports uncertainty and risk-aware predictions
Cons	More complex than point metrics
Best use cases	Forecast intervals and asymmetric costs

Practical Metric Selection Guide

Situation	Recommended Metric
Need average error in original units	MAE
Large errors are very costly	RMSE or MSE
Need training loss for smooth optimization	MSE
Need robust training with outliers	Huber loss
Need model fit relative to mean baseline	R^2
Comparing linear models with feature counts	Adjusted R^2
Need percentage business error	MAPE or WMAPE
Need prediction intervals	Quantile loss

Combined Python Example

```
import numpy as np
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
)

def regression_report(y_true, y_pred, num_features=None):
    y_true = np.asarray(y_true, dtype=float)
    y_pred = np.asarray(y_pred, dtype=float)

    mae = mean_absolute_error(y_true, y_pred)
    mse = mean_squared_error(y_true, y_pred)
    rmse = mean_squared_error(y_true, y_pred, squared=False)
    r2 = r2_score(y_true, y_pred)

    report = {
        "MAE": mae,
        "MSE": mse,
        "RMSE": rmse,
        "R2": r2,
    }

    if num_features is not None:
        n = len(y_true)
        if n > num_features + 1:
            report["Adjusted R2"] = 1 - ((1 - r2) * (n - 1) / (n - num_features -
1))

    nonzero_mask = y_true != 0
    if np.any(nonzero_mask):
        report["MAPE"] = (
            np.mean(np.abs((y_true[nonzero_mask] - y_pred[nonzero_mask]) /
y_true[nonzero_mask]))
```

```
        * 100
    )

    return report

y_true = [100, 200, 300, 400, 500]
y_pred = [110, 190, 330, 390, 480]

for name, value in regression_report(y_true, y_pred, num_features=2).items():
    print(f"{name}: {value:.4f}")
```

Final Interview Advice

Do not say one regression metric is always best. The correct answer depends on target scale, outliers, business cost, and whether the goal is point prediction, robust training, model comparison, or uncertainty estimation.

Strong interview answer:

"I would report at least one interpretable error metric such as MAE or RMSE, check R^2 for baseline-relative fit, inspect residuals, and choose the primary metric based on the business cost of different errors."